JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.channels.spi

Class AbstractInterruptibleChannel

java.lang.Object
  java.nio.channels.spi.AbstractInterruptibleChannel

All Implemented Interfaces:

`Closeable`, `AutoCloseable`, `Channel`, `InterruptibleChannel`

Direct Known Subclasses:

`FileChannel`, `SelectableChannel`

```
public abstract class AbstractInterruptibleChannel
extends Object
implements Channel, InterruptibleChannel
```

Base implementation class for interruptible channels.

This class encapsulates the low-level machinery required to implement the asynchronous closing and interruption of channels. A concrete channel class must invoke the `begin` and `end` methods before and after, respectively, invoking an I/O operation that might block indefinitely. In order to ensure that the `end` method is always invoked, these methods should be used within a `try` ... `finally` block:

```
boolean completed = false;
try {
    begin();
    completed = ...;    // Perform blocking I/O operation
    return ...;         // Return result
} finally {
    end(completed);
}
```

The `completed` argument to the `end` method tells whether or not the I/O operation actually completed, that is, whether it had any effect that would be visible to the invoker. In the case of an operation that reads bytes, for example, this argument

should be `true` if, and only if, some bytes were actually transferred into the invoker's target buffer.

A concrete channel class must also implement the `implCloseChannel` method in such a way that if it is invoked while another thread is blocked in a native I/O operation upon the channel then that operation will immediately return, either by throwing an exception or by returning normally. If a thread is interrupted or the channel upon which it is blocked is asynchronously closed then the channel's `end` method will throw the appropriate exception.

This class performs the synchronization required to implement the `Channel` specification. Implementations of the `implCloseChannel` method need not synchronize against other threads that might be attempting to close the channel.

Since:

`1.4`

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**AbstractInterruptibleChannel**()	Initializes a new instance of this class.

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description
protected void	**begin**()	Marks the beginning of an I/O operation that might block indefinitely.
void	**close**()	Closes this channel.
protected void	**end**(boolean completed)	Marks the end of an I/O operation that might block indefinitely.
protected abstract void	**implCloseChannel**()	

		Closes this channel.
boolean	**isOpen**()	Tells whether or not this channel is open.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

AbstractInterruptibleChannel

```
protected AbstractInterruptibleChannel()
```

Initializes a new instance of this class.

Method Detail

close

```
public final void close()
                throws IOException
```

Closes this channel.

If the channel has already been closed then this method returns immediately. Otherwise it marks the channel as closed and then invokes the `implCloseChannel` method in order to complete the close operation.

Specified by:

`close` in interface `Closeable`

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Channel`

Specified by:

`close` in interface `InterruptibleChannel`

Throws:

`IOException` - If an I/O error occurs

implCloseChannel

```
protected abstract void implCloseChannel()
                                    throws IOException
```

Closes this channel.

This method is invoked by the `close` method in order to perform the actual work of closing the channel. This method is only invoked if the channel has not yet been closed, and it is never invoked more than once.

An implementation of this method must arrange for any other thread that is blocked in an I/O operation upon this channel to return immediately, either by throwing an exception or by returning normally.

Throws:

`IOException` - If an I/O error occurs while closing the channel

isOpen

```
public final boolean isOpen()
```

Description copied from interface: `Channel`

Tells whether or not this channel is open.

Specified by:

`isOpen` in interface `Channel`

Returns:

`true if, and only if, this channel is open`

begin

```
protected final void begin()
```

Marks the beginning of an I/O operation that might block indefinitely.

This method should be invoked in tandem with the `end` method, using a `try` ... `finally` block as shown above, in order to implement asynchronous closing and interruption for this channel.

end

```
protected final void end(boolean completed)
                  throws AsynchronousCloseException
```

Marks the end of an I/O operation that might block indefinitely.

This method should be invoked in tandem with the `begin` method, using a `try` ... `finally` block as shown above, in order to implement asynchronous closing and interruption for this channel.

Parameters:

`completed - true if, and only if, the I/O operation completed successfully, that is, had some effect that would be visible to the operation's invoker`

Throws:

`AsynchronousCloseException - If the channel was asynchronously closed`

`ClosedByInterruptException - If the thread blocked in the I/O operation was interrupted`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.